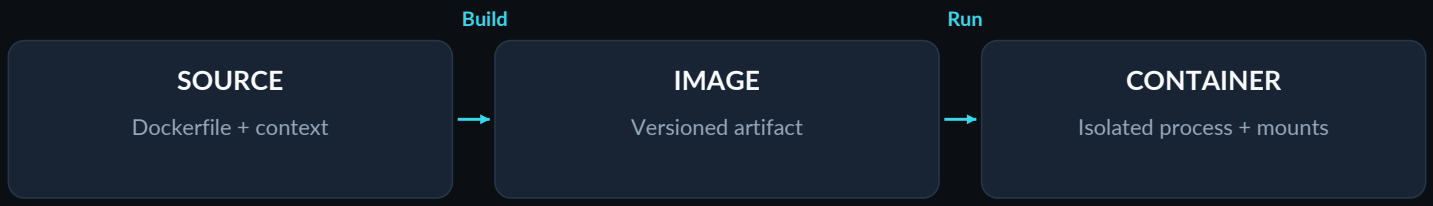
DOCKER

Build once. Understand every layer.

A dense, practical reference for learning, shipping, debugging and revising containerized applications.

LINUX CONTAINERS | MODERN COMPOSE | BUILDKIT



CONTENTS

Every row is clickable

PICK YOUR ROUTE

New to Docker? Start on p2, then build the app on p6.
Shipping today? Use p9-12. **Revising?** Jump to p15-16.

18 pages / Selectable code / Linked references / Checked 07 Sep 2026

Foundations & architecture

The mental model: a container is a process with boundaries, not a tiny virtual machine.

MUST KNOW

Docker packages an application and its user-space dependencies into an image, then runs isolated instances called containers. It reduces environment drift; it does not remove kernel, CPU-architecture, network or data dependencies.

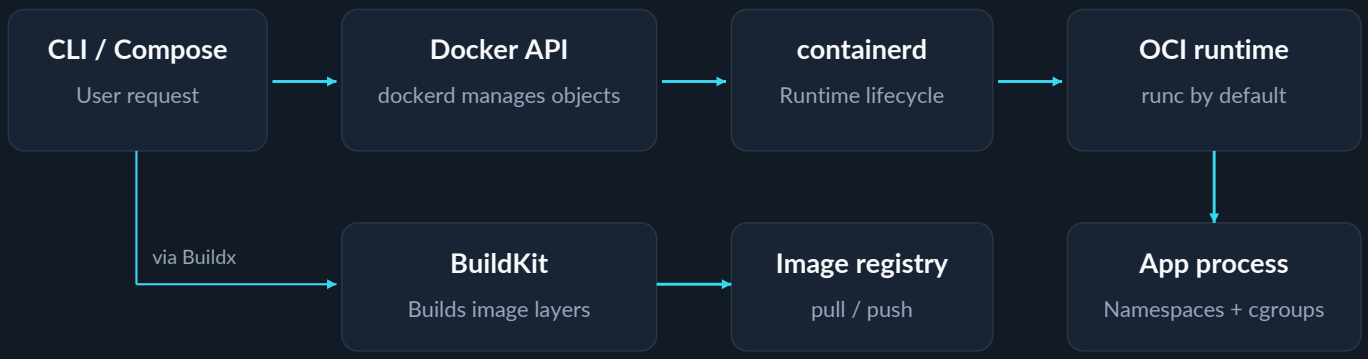
The four nouns

Image — immutable template.
Container — runtime instance + writable state.
Registry — distribution service for images.
Volume — data with an independent lifecycle.

The two boundaries

Namespaces isolate views of processes, networking and mounts.
cgroups account for and limit resources.
Capabilities + seccomp narrow privileges and allowed system calls.

HOW A COMMAND BECOMES A CONTAINER



Runtime path above; build/distribution path below. On Docker Desktop, Linux containers run inside a Linux VM.

Containers vs virtual machines

DIMENSION	CONTAINER	VIRTUAL MACHINE
Isolation	Shares the running Linux kernel	Separate guest kernel per VM
Payload	App + runtime + required libraries	Guest OS + app + dependencies
Trade-off	Low overhead; shared-kernel risk	Stronger OS boundary; more overhead
Choose when	Portable app/service packaging	Need another kernel or stronger isolation

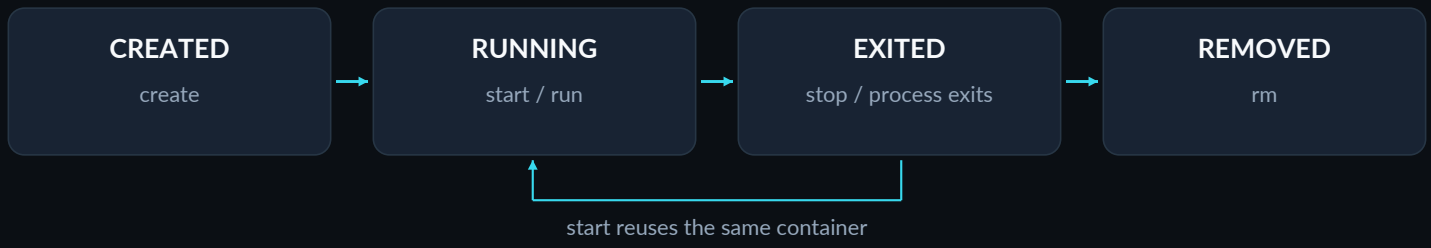
REMEMBER

Image = recipe; container = a running dish from that recipe. The analogy stops at the kernel: Linux images carry user-space files, not a separate booted Linux kernel.

OFFICIAL REFERENCES S01 S02 S03 S16 S35 S42 Full source index p18

Container lifecycle & CLI

Create, run, inspect, stop and replace: know which object each command changes.



Daily commands

COMMAND	WHAT IT DOES
`docker ps -a`	List running + stopped
`docker run IMAGE`	Create + start a new instance
`docker start app`	Start an existing instance
`docker exec -it app sh`	New process inside running app
`docker logs -f app`	Follow stdout/stderr logs
`docker stop -t 20 app`	Graceful stop; 20 s timeout
`docker rm app`	Remove stopped container
`docker stats app`	Live resource usage

Flags that matter

FLAG	USE / CAUTION
`-d`	Detached; process must stay alive
`-it`	Interactive stdin + terminal
`--name app`	Human-readable container name
`--rm`	Remove on exit; disposable jobs
`-p 8080:3000`	Host port : container port
`-e MODE=prod`	Runtime environment variable
`--env-file .env`	Read runtime variables from file
`--init`	Small init; forwards signals/reaps

Process & signal rules

PID 1 controls lifetime. A container exits when its main process exits. Detached is not immortal.

stop: configured stop signal (default SIGTERM), then SIGKILL after timeout. **kill:** SIGKILL by default.

exec vs attach: exec creates a process; attach joins the existing main process streams.

Pause / unpause: freeze / resume processes; neither recreates the container.

Restart policy: choose deliberately

POLICY	BEHAVIOR
`no`	Default: no automatic restart
`on-failure:3`	Retry non-zero exits, up to 3 times
`always`	Restart; manual stop lasts until daemon restart
`unless-stopped`	Like always, but preserves explicit stop

USEFUL SHORTCUT

run = new · start = same · exec = extra.
To detach from an interactive (-it) session without stopping it: **Ctrl+P, then Ctrl+Q**.

OFFICIAL REFERENCES S03 S04 S05 S06 S07

Full source index p18

Images, layers & registries

Build artifacts are immutable; their tags are not. Treat release identity as a first-class decision.

ONE IMAGE / MANY RUNTIME INSTANCES

Copy-on-write: changing a lower-layer file creates a writable copy. Removing a container deletes its writable layer, not a named volume.

Image operations

COMMAND	PURPOSE
`docker image ls`	List local images
`docker pull IMAGE`	Fetch from registry
`docker build -t demo:1 .`	Build from current context
`docker image inspect IMAGE`	Config, IDs and metadata
`docker history IMAGE`	Layer/build history
`docker tag demo:1 USER/demo:1`	Add a reference; not a copy
`docker push USER/demo:1`	Upload to a writable repository
`docker image rm demo:1`	Remove local reference/image

Name, tag, digest, platform

`registry.example.com/team/api:1.4`
Registry / namespace / repository : tag.

Tag: movable label. `latest` is a conventional default tag, not "newest" or a production guarantee.

Digest: content-addressed identity. Deploy `repo@sha256:...` with a real verified digest; pin and update deliberately.

Image ID is not the registry digest. A multi-platform index selects the matching OS/CPU image manifest.

Move an image offline

BASH

```
docker image save -o demo.tar demo:1
docker image load -i demo.tar
```

Result: transfers image layers and configuration. Does not back up database volumes.

save/load vs export/import

Choose save/load to preserve a runnable image archive with its metadata.

Choose export/import only for a flattened filesystem snapshot. Export excludes mounted-volume data and loses the original image history/configuration.

RELEASE RULE

Use `docker login` to push. Build once, test, then promote one digest through all environments.

OFFICIAL REFERENCES S08 S09 S10 S11 S14 S40 S48

Full source index p18

Dockerfile syntax & startup

Dockerfile = how to build the image; runtime options = how to launch its process.

BUILD TIME — RUN / COPY change files → **IMAGE DEFAULTS** — ENV / USER / CMD etc. → **RUNTIME** — docker run overrides

The instruction vocabulary

INSTRUCTION / EXAMPLE	ROLE	IMPORTANT RULE
`FROM node:24-bookworm-slim`	Base / new stage	Each FROM starts a stage; name with AS.
`WORKDIR /app`	Working directory	Prefer absolute paths; creates missing directories.
`COPY src/ ./src/`	Copy build inputs	Only read sources inside the build context.
`RUN npm ci`	Build-time command	Executes at build, not container startup.
`ARG APP_VERSION=1`	Build variable	Build scope only; not a secret channel.
`ENV NODE_ENV=production`	Persistent default	Runtime default; can be overridden.
`USER node`	Execution identity	Sets UID; verify required file permissions.
`EXPOSE 3000`	Port metadata	Metadata only; does not publish a port.
`ENTRYPOINT ["node"]`	Default executable	Executable + CMD arguments; use JSON.
`CMD ["server.js"]`	Default command/args	Default args; runtime args replace CMD.
`HEALTHCHECK CMD ...`	Health probe	Probe exit 0 = healthy; 1 = unhealthy.
`COPY --from=build ...`	Stage artifact transfer	Copy required artifacts into the final stage.
`LABEL key="value"`	Image metadata	Attach owner, version or source metadata.
`STOPSIGNAL SIGTERM`	Shutdown signal	Default SIGTERM; app must handle it.

ENTRYPOINT + CMD: assemble the final process

IMAGE SETUP	RUN-TIME INPUT	EFFECTIVE PROCESS
`ENTRYPOINT ["node"]` `CMD ["server.js"]`	`docker run demo:1`	`node server.js`
Same explicit setup	`docker run demo:1 other.js`	`node other.js`
Override executable	`docker run --entrypoint sh demo:1 -c "id"`	`sh -c "id"`

SHELL vs EXEC

`CMD ["echo", "$HOME"]` prints literal `$HOME`. JSON form performs no shell expansion. Use a shell explicitly only when needed; use `exec` in wrapper scripts.

GOOD TO KNOW

COPY vs ADD: prefer COPY for ordinary local files. ADD adds remote-fetch/archive behavior.
VOLUME: declares a mount point, not a backup or a host directory path.

OFFICIAL REFERENCES S03 S12 S13 S14 S36

Full source index p18

Build a working application

A complete Node.js lab: three small files, no npm packages, and a real health probe.

1 / server.js — JAVASCRIPT

```
const http = require("node:http");
const server = http.createServer((req,res)=>{
  res.setHeader("Content-Type","text/plain");
  res.end(req.url === "/health"
    ? "ok\n" : "hello docker\n");
});
server.listen(3000, "0.0.0.0");
process.on("SIGTERM", () => {
  server.close(() => process.exit(0));
  setTimeout(() => process.exit(1),
    8000).unref();
});
```

Result: serves HTTP on port 3000. SIGTERM drains the server; the timeout bounds shutdown.

2 / healthcheck.js — JAVASCRIPT

```
fetch("http://127.0.0.1:3000/health")
  .then(r => process.exit(r.ok ? 0 : 1))
  .catch(() => process.exit(1));
```

Result: exit 0 for a successful probe, otherwise 1. Probe tools must exist inside the image.

3 / Dockerfile — DOCKERFILE

```
# syntax=docker/dockerfile:1
FROM node:24-bookworm-slim
WORKDIR /app
ENV NODE_ENV=production
COPY --chown=node:node *.js ./
USER node
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=3s \
  --start-period=5s --retries=3 \
  CMD ["node", "healthcheck.js"]
CMD ["node", "server.js"]
```

Result: a non-root image with an exec-form startup command. The versioned base tag still moves; pin a reviewed digest for a release.

Add .dockerignore — REFERENCE

```
.git
node_modules
.env
.env.*
secrets/
*.log
```

Why: avoid sending credentials and irrelevant files to the build. This is not a substitute for keeping secrets out of source control.

Build, run, verify — BASH

```
docker build -t demo:1 .
docker run -d --name app --init \
  -p 127.0.0.1:8080:3000 demo:1
curl http://127.0.0.1:8080
# hello docker
docker inspect --format \
  '{{.State.Health.Status}}' app
# starting, then healthy
```

Assumes: Docker is running; files share one directory; port 8080 and name app are free.

What this lab teaches

Bind inside to 0.0.0.0. Binding only to the container's loopback blocks published-port access.

Publish outside to 127.0.0.1. This keeps the demo local rather than exposing all host interfaces.

Replace, do not hand-edit. Change source, rebuild the image, then recreate the container.

Cleanup: `docker stop app` then `docker rm app`. No persistent volume is used.

Replace the container after changing server.js — BASH

```
docker stop app && docker rm app
docker build -t demo:2 .
docker run -d --name app --init -p 127.0.0.1:8080:3000 demo:2
```

Expected: requests now reach the rebuilt app. Starting the old container would still use its original image.

OFFICIAL REFERENCES S03 S06 S12 S13 S14 S15

Full source index p18

Storage & persistence

A container can be disposable without making its data disposable.

Choose the right storage boundary

TYPE	SURVIVES CONTAINER REMOVAL?	BEST FIT / LIMITATION
Writable layer	No; retained across stop/start only	Disposable scratch files; copy-on-write overhead
Named volume	Yes, until explicitly removed	Application data; Docker-managed, usually host-local
Bind mount	Yes, in the host source path	Source code/config; host path and permissions matter
tmpfs (Linux)	No; lost when container stops	Ephemeral memory-backed data; may be swapped

PERSISTENT APP DATA? Named volume → **EDIT FROM HOST?** Bind mount → **TEMPORARY ONLY?** tmpfs

Prove volume persistence

BASH

```
docker volume create demo-data
docker run --rm -v demo-data:/data \
  alpine:3.23 sh -c 'echo kept >/data/n'
docker run --rm -v demo-data:/data \
  alpine:3.23 cat /data/n
# kept
```

Result: the first container is gone; the named volume still contains the file.

Read-only bind mount

BASH

```
docker run --rm --mount \
  type=bind,src="$PWD",dst=/src,readonly \
  alpine:3.23 ls /src
```

Result: lists your current host directory inside /src. POSIX shell; paths are resolved on the daemon host.

Mount behavior that surprises people

Mounts hide existing files. Mounting a host directory over /app can hide the app or its installed dependencies.

New empty volumes can be seeded from the image's existing directory. Non-empty volumes hide image contents; `volume-nocopy` disables initial copying.

Permissions are numeric UID/GID. A matching username is not enough. Match ownership instead of using `chmod 777`.

--mount vs -v: a missing bind source normally errors with --mount; -v may create a directory. Prefer the explicit form.

Backup is a separate operation

A named volume is **not** a backup, replication or cross-host storage. Use database-aware exports or stop/quiesce writers before a consistent filesystem backup. Rehearse restore, not only backup.

Safe inspection & removal

```
docker volume ls
docker volume inspect demo-data
```

After confirming the demo data is disposable:

```
docker volume rm demo-data
```

VOLUME LIFECYCLE

Named volumes survive --rm. Anonymous volumes created with --rm are cleaned up on container exit.

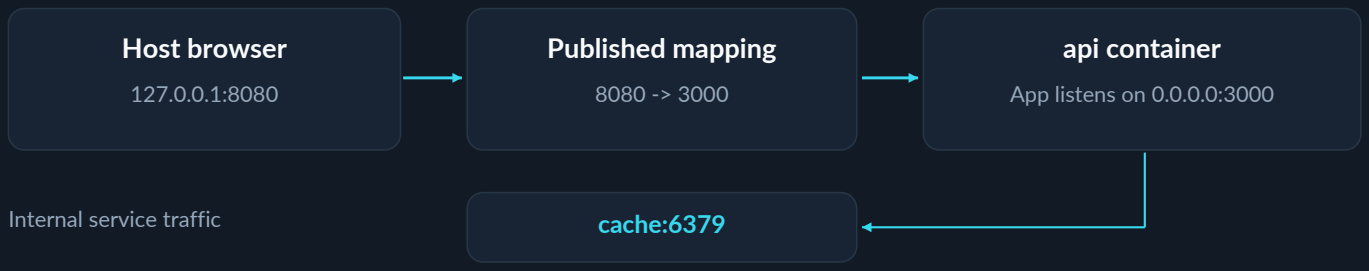
OFFICIAL REFERENCES S09 S16 S17 S18 S19 S52

Full source index p18

Networking & ports

Think in two address spaces: the host's ports and the container network's ports.

FOLLOW THE PACKET



Port facts

SYNTAX / CONCEPT	ACTUAL MEANING
`-p 8080:3000`	Publish container 3000 on host 8080
`-p 127.0.0.1:8080:3000`	Bind only the host loopback address
`-p 5353:53/udp`	UDP mapping; TCP is the default
`EXPOSE 3000`	Metadata, not port publishing
Same network	Use service DNS + container port; no -p needed

Network drivers

DRIVER	CHOOSE WHEN
User-defined bridge	Normal single-host app networks; built-in name resolution
host	Share host network; no port translation / -p
none	No normal external networking
overlay	Multi-host container networks (Swarm)
macvlan / ipvlan	Special LAN/IP integration; requires network planning

DNS without publishing Redis

BASH

```
docker network create lab
docker run -d --rm --name cache \
  --network lab redis:8-alpine
docker run --rm --network lab \
  redis:8-alpine redis-cli -h cache ping
# PONG
```

Result: the client resolves cache on lab. Cleanup: `docker stop cache` then `docker network rm lab`.

localhost means "this namespace"

Inside api: localhost is api itself, not Redis and not your laptop. Use `cache:6379` for a sibling service.

To the host on Desktop: use `host.docker.internal`. On Linux Engine, add `--add-host host.docker.internal:host-gateway` when needed.

Host mode: native Linux support; Docker Desktop support is opt-in on supported versions. Published-port flags are ignored in host mode.

SECURITY BOUNDARY

Publishing without a host IP normally binds all host interfaces. Keep databases unpublished by default; network separation is not authentication. Review host firewall/routing behavior before exposing a service.

OFFICIAL REFERENCES S20 S21 S22 S23 S24 S27 S51 Full source index p18

Compose: a complete stack

Declare services, networks and volumes together; Compose manages their lifecycle as one project.

compose.yaml

YAML

```yaml
name: docker-lab
services:
  api:
    build: .
    ports:
      - "127.0.0.1:8080:3000"
    init: true
    depends_on:
      cache:
        condition: service_healthy
    environment:
      REDIS_URL: redis://cache:6379
    restart: unless-stopped
  cache:
    image: redis:8-alpine
    command:
      - redis-server
      - --appendonly
      - "yes"
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 2s
      retries: 10
volumes:
  redis-data:
```

Prerequisite: use the files from p6. Stop/remove the standalone app first so port 8080 is free.

Prefix every command: docker compose

COMMAND	RESULT
`config`	Render/validate resolved model
`up -d --build --wait`	Build, create/start; wait for running/healthy
`ps`	List project containers
`logs -f --tail=100`	Follow recent service logs
`exec api sh`	Open a shell in running api
`stop`	Stop; keep containers/network
`down`	Remove project containers/network; keep named volumes
`down -v`	Also remove declared named and attached anonymous volumes.

Startup order is not readiness

Short depends_on: dependency is started, not necessarily ready.
service_healthy: wait for its health check.
service_completed_successfully: useful for a one-off migration job.

Add retries anyway: dependencies can fail after startup.

Run the stack and test Redis

BASH

```bash
docker compose up -d --build --wait
docker compose exec cache redis-cli ping
# PONG
curl http://127.0.0.1:8080/health
# ok
```

Result: both services start; Redis stores AOF data in redis-data. No Redis host port is published.

LAB BOUNDARIES

The p6 API is intentionally dependency-free: it does **not** use REDIS_URL yet. Adding a Redis client + retry policy is a practice task.

This unauthenticated Redis configuration is for a local lab, not a production database deployment.

Prove data survives stack replacement

BASH

```bash
docker compose exec cache redis-cli set lesson retained
docker compose down && docker compose up -d --wait
docker compose exec cache redis-cli get lesson
# "retained"
```

Expected: the stored value remains. Do not use down -v when you need the volume.

OFFICIAL REFERENCES S25 S26 S27 S28 S29 S30 S51

Full source index p18

Configuration & secrets

Separate template substitution, container environment and sensitive runtime inputs.

TWO DIFFERENT VARIABLE SYSTEMS

Shell / .env / --env-file
Compose interpolates ${VAR}

→

Resolved compose.yaml
Explicit environment / env_file

→

Container environment
What the application receives

Explicit runtime configuration

YAML

```yaml
services:
  api:
    build: .
    environment:
      MODE: "${MODE:-development}"
    env_file:
      - ./app.env
```

Result: MODE is set explicitly; additional variables load from app.env. An .env file alone does not inject every variable into the container.

Runtime secret overlay

YAML

```yaml
services:
  api:
    secrets:
      - api_token
secrets:
  api_token:
    file: ./secrets/api_token.txt
```

Save as: compose.secrets.yaml. Create the file locally; exclude secrets/ from Git and the build context. Mounted at /run/secrets/api_token.

Precedence: resolve, then inject

For common explicit settings, higher wins:
run -e → environment → env_file → image ENV.

Shell and .env values feed interpolation first. Bare keys and interpolated values need that extra resolution step. Use `docker compose config` to verify the result.

A MOUNT IS NOT A SECRET VAULT

Local Compose file-backed secrets are mounted from the host; they are not automatically encrypted at rest. Secure the host file and access.

The app must read the secret file. A `_FILE` environment convention only works when that image/app implements it.

Apply the overlay, inspect the model, then recreate

BASH

```
docker compose -f compose.yaml -f compose.secrets.yaml config
docker compose -f compose.yaml -f compose.secrets.yaml up -d
```

Result: api receives its declared secret mount. Resolved config and environment output can reveal sensitive values; do not paste them publicly.

Project & environment isolation

`docker compose -p qa up -d` gives a separate project namespace. It does not solve fixed host-port collisions.

Use ordered `-f` overlays for dev/prod; inspect the merged model. Profiles can enable optional tools.

Modern Compose rules

Use `docker compose` and the Compose Specification (v2/v5 CLIs). Omit the obsolete top-level `version:`.

`restart` reuses container configuration; `up -d` reconciles changed service configuration.

OFFICIAL REFERENCES S25 S30 S31 S32 S33 S34 S39 Full source index p18

BuildKit, cache & delivery

Fast builds come from stable inputs and reusable work; small runtime images come from separation.

Multi-stage npm application

DOCKERFILE

```
FROM node:24-bookworm AS build
WORKDIR /app
COPY package*.json ./
RUN --mount=type=cache,target=/root/.npm \
  npm ci
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:24-bookworm-slim
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/package.json ./
COPY --from=build /app/node_modules \
  ./node_modules
COPY --from=build /app/dist ./dist
USER node
CMD ["node","dist/server.js"]
```

Assumes: a lockfile, production dependencies and a build script that emits dist/server.js. Test native-library compatibility in the final image. This is a different app pattern from p6.

Cache rules worth remembering

Stable first, volatile last: copy lockfiles, install dependencies, then copy frequently changing source.

Keep context small. Cache mounts reuse package downloads; external cache helps ephemeral CI runners.

Cache is not freshness. RUN does not re-run just because packages changed upstream. `--no-cache` does not refresh the base; add `--pull` when needed.

Secret value changes do not invalidate cache. Invalidate the affected step deliberately after rotation.

Build credentials, not image layers

REFERENCE

```
RUN --mount=type=secret,id=npmrc \
  NPM_CONFIG_USERCONFIG=/run/secrets/npmrc \
  npm ci

# Pass from the build client:
docker build --secret \
  id=npmrc,src="$HOME/.npmrc" .
```

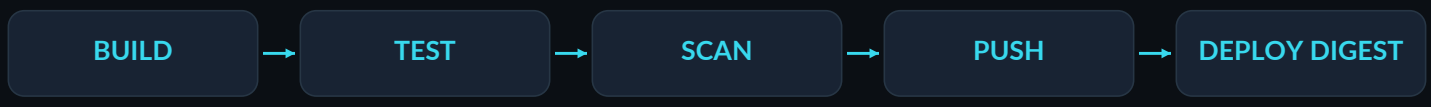
Use: the RUN stanza in the Dockerfile; the docker command in your shell. Never copy or print the mounted secret.

Multi-platform CI build + reusable registry cache

BASH

```
docker buildx create --name multi --driver docker-container --use
docker buildx build --platform linux/amd64,linux/arm64 \
  -t REGISTRY/USER/api:1.0 \
  --cache-from type=registry,ref=REGISTRY/USER/api:buildcache \
  --cache-to type=registry,ref=REGISTRY/USER/api:buildcache,mode=max \
  --sbom=true --provenance=true --push .
```

Replace REGISTRY/USER and authenticate first. Requires native builders, emulation or cross-compilation for the targets. Result is pushed, not automatically loaded locally; use --load where supported. Attestations require compatible storage.



PERFORMANCE TRADE-OFF

Native builders avoid emulation overhead. Keep build tools out of the final stage: deleting them in a later layer does not remove their earlier-layer bytes.

OFFICIAL REFERENCES S13 S14 S35 S36 S37 S38 S39 S40 S41

Full source index p18

Security & reliability

Reduce privileges, bound resource usage and make failures observable and recoverable.

CRITICAL BOUNDARY

Access to a rootful Docker daemon/socket is effectively host-administrator access. Do not mount docker.sock into untrusted workloads, expose an unauthenticated API, or “fix” permissions with world-writable socket access.

Best-practices checklist

Run the app as non-root. Remove capabilities; keep default seccomp/AppArmor/SELinux protections where available.

Use read-only filesystems where possible; allow writes only to required volumes or tmpfs paths.

Keep secrets outside images. Use build secret mounts and controlled runtime secret delivery.

Trust and maintain the artifact. Pin reviewed digests, rebuild for patches, scan dependencies and retain provenance.

Limit blast radius. Restrict published interfaces and API access; isolate unrelated services.

Test restore and rollback. Keep durable data outside writable container layers; validate schema compatibility.

Harden the p6 demo

BASH

```
docker run -d --name hardened --init \
  -p 127.0.0.1:8081:3000 \
  --read-only \
  --tmpfs /tmp:rw,noexec,nosuid \
  --cap-drop=ALL \
  --security-opt no-new-privileges=true \
  --memory=256m --cpus=1 --pids-limit=100 \
  --restart=unless-stopped \
  --log-driver=local demo:1
```

Result: a constrained non-root demo on port 8081. Real apps may need explicit writable paths or narrowly selected capabilities; test before rollout.

Limits are not capacity planning

CONTROL	MEANING
`--memory`	Memory cap; exhaustion can kill the workload
`--cpus`	CPU-time quota; throttling can raise latency
`--pids-limit`	Task limit, including threads
`--log-driver=local`	Local driver with rotation by default

USER vs rootless daemon

USER: reduces the application process’s privileges inside its container.

Rootless mode: runs the daemon and containers without host root. Check cgroup, networking and filesystem constraints for your deployment.

Three reliability rules

Healthy is a signal, not recovery. Plain restart policies react to exits, not health status alone.
Shutdown must be graceful. Handle SIGTERM, drain work and exit before the grace period.
Logs must be bounded. Emit stdout/stderr and configure retention/rotation.

AVOID

Avoid `--privileged`, broad host mounts, secrets in ARG/ENV, manual fixes inside running containers and unbounded logs. Smaller images reduce shipped components; they do not prove the app is secure.

OFFICIAL REFERENCES S03 S07 S12 S17 S42 S43 S44 S45 S46 S47

Full source index p18

Debugging & common pitfalls

Observe before changing anything. Find whether the failure is in the process, filesystem or network.

PS → LOGS → INSPECT → NETWORK → RESOURCES

Symptom → likely cause → first check

SYMPTOM	LIKELY CAUSE	FIRST CHECK
Exits immediately	Main command finished or crashed	Logs; exit code; effective entrypoint/CMD
Cannot reach HTTP app	Wrong bind address, port or network	App listener; docker port; service DNS
Files missing / permission denied	Mount hides path; wrong UID/GID	Inspect mounts; check numeric ownership
No space / killed under load	Logs, images/cache or memory limit	docker system df; stats; OOMKilled state
exec format error / command missing	Wrong architecture, shebang or binary	Image platform; interpreter; line endings

Minimal diagnostic sequence

BASH

```
docker ps -a
docker logs --tail 100 app
docker inspect --format \
  '{{json .State}}' app
docker port app
docker stats --no-stream app
```

Result: status, output, exit/OOM/health state, port mappings and current resource use. Inspect before deleting evidence.

Exit codes: evidence, not diagnosis

CODE	INTERPRETATION
0 / non-zero	Success / application-specific failure
125 / 126 / 127	Docker run error / cannot execute / not found
137 = 128 + 9	SIGKILL convention; OOM is only one possible cause
143 = 128 + 15	SIGTERM convention; may be normal shutdown

Common Mistakes & Pitfalls

WRONG / WHY IT FAILS	CORRECT / WHY IT WORKS
× EXPOSE makes the app public. It is only image metadata.	Add an intentional -p mapping. Bind the app to its container interface.
× localhost reaches another service. It points back to this namespace.	Use service DNS + container port on a shared user-defined network.
× depends_on means ready forever. Startup order is not ongoing availability.	Gate on health; retry at runtime. Dependencies can fail after startup.
× restart applies changed environment. It reuses container configuration.	Use compose up -d to reconcile. Rebuild when image inputs changed.
× Delete a secret in a later layer. Earlier layers can still contain it.	Use a build secret mount. Never copy it into an image layer.
× “No bash” means Docker is broken. Minimal images often omit shells/tools.	Use available tools or a debug image. Do not permanently modify production.

OFFICIAL REFERENCES S03 S04 S05 S06 S12 S17 S21 S26 S30 S38 S46

Full source index p18

Patterns, scale & interviews

Docker packages and runs workloads; production architecture still has to manage traffic, state and failure.

SCALE STATELESS WORK; DESIGN STATE SEPARATELY

Choose the control layer

TOOL	BEST USE / TRADE-OFF
Docker run	One container or small explicit script; manual coordination
Compose	A declared multi-service app on an Engine; not a multi-host scheduler
Swarm / Kubernetes	Cluster scheduling and reconciliation; more operational complexity

Choose the runtime base

BASE	MAIN TRADE-OFF
Full distro image	More tools/dependencies; larger artifact
Slim Debian variant	Smaller glibc base; install required libraries
Alpine variant	Compact musl base; native compatibility can differ
Distroless / scratch	Minimal or empty base; few/no debug tools; supply required runtime assets

Interview essentials

Image vs container? Immutable artifact vs a runtime instance with its own state and configuration.

Namespaces vs cgroups? What a process can see vs how much it may consume.

One process per container? Prefer one main service responsibility; workers or child processes are valid.

Always portable? Only across compatible kernel/architecture/runtime assumptions, or with suitable emulation.

Why not docker commit releases? Hand-mutated snapshots hide the reproducible build recipe; keep a Dockerfile.

SBOM vs provenance? What is inside the artifact vs how it was built. Neither alone is a vulnerability verdict.

SCALING GOTCHA

`docker compose up -d --scale api=3` needs a compatible service definition. Remove fixed host-port conflicts and `container_name`; put a proxy in front. Named local volumes do not become replicated storage.

Use a remote Engine without exposing an unauthenticated API

BASH

```
docker context create staging --docker "host=ssh://user@host"
docker --context staging ps
docker context show
```

Replace user@host with your authorized SSH target. The first command saves a context; the second queries that Engine. Check the active context before any destructive command.

OFFICIAL REFERENCES S01 S15 S17 S25 S40 S41 S49 S50 S53

Full source index p18

Quick reference

A 5-10 minute revision route: identify the object, choose the command, check the boundary.

Know what you are asking Docker

QUESTION	COMMAND
Client + server versions?	`docker version`
Engine details?	`docker info`
Which remote target?	`docker context show`
Compose / builder installed?	`docker compose version` `docker buildx version`
What is running / stopped?	`docker ps` / `docker ps -a`
What image was built?	`docker image inspect IMAGE`
Where is storage used?	`docker system df -v`
Which network / mounts?	`docker inspect app`

Choose the right boundary

NEED	DEFAULT CHOICE
Runtime instance	docker run; disposable job: --rm
Extra process	exec; not attach
Durable application data	Named volume; backups separately
Editable host source	Bind mount; narrow permissions
Temporary scratch data	tmpfs on Linux
Service-to-service traffic	User-defined network + service DNS
Host/browser traffic	Explicit -p HOST:CONTAINER
App stack definition	compose.yaml; docker compose up

Rules that prevent most bugs

run = new · start = same · exec = extra.

H:C = Host:Container. In `-p 8080:3000`, your browser uses 8080; peers use 3000.

RUN builds; CMD starts. ENTRYPOINT sets the executable; CMD provides replaceable defaults.

Image tag ≠ immutable release. Use a reviewed digest to identify deployed content.

Stop ≠ remove ≠ delete volume. Treat each lifecycle as a separate decision.

Practical command patterns

`docker logs -f --tail 100 app`
Bound the initial log output, then follow.

`docker exec -it app sh`
Interactive shell only when one exists.

`docker cp app:/path/file ./file`
Retrieve a file; not a database backup.

`docker compose exec -T api COMMAND`
Disable TTY allocation for noninteractive CI.

`docker COMMAND --help`
Check installed-version flags before scripting.

Cleanup ladder: inspect → targeted removal → broader cleanup

COMMAND	SCOPE / RISK
`docker rm app`	Remove one stopped container and its writable layer.
`docker image prune`	Remove dangling images; -a expands to all unused images.
`docker system prune`	Stopped containers, unused networks, dangling images and unused build cache.
`docker volume rm NAME`	Explicit data deletion. Confirm identity and backup first.

DESTRUCTIVE FLAGS · system prune --volumes adds eligible anonymous volumes. volume prune defaults to unused anonymous volumes; -a includes named ones. “Unused” ≠ “unimportant.”

OFFICIAL REFERENCES S03 S04 S08 S12 S25 S28 S29 S48 S50 S54 S55 Full source index p18

One-page master cheatsheet

Read left to right. Replace IMAGE / COMMAND / NAME with your actual values. Linux + modern Compose.

DOCKERFILE + CONTEXT → IMAGE / DIGEST → CONTAINER + CONFIG

01 / Mental model & boundaries

Image: immutable app + user-space files.
Container: process instance + writable layer.
Volume: persistent data, separate lifecycle.
Registry: publishes/pulls image artifacts.
Namespaces: isolation. **cgroups:** limits.
Tag: movable. **Digest:** content identity.
Linux kernel: shared; Desktop uses a VM.
run = new · start = same · exec = extra.

02 / Launch & inspect

BASH

```
docker run -d --name app \
  -p 127.0.0.1:8080:3000 IMAGE
docker ps -a
docker logs -f --tail 100 app
docker exec -it app sh
docker stop app
docker rm app
```

-d: detached · **-it:** interactive terminal.
--rm: disposable · **--init:** init helper.
Assumes the image serves container port 3000.

03 / Build & startup rules

```
docker build -t app:1 .
docker build --pull --no-cache .
```

FROM → WORKDIR → COPY → RUN
USER → EXPOSE → CMD (typical structure)
RUN executes at build; CMD is runtime default.
ENTRYPOINT + CMD = executable + default args.
Prefer exec JSON; last CMD/ENTRYPOINT wins.
Lockfiles first; source last; use .dockerignore.
Multi-stage: copy only runtime artifacts.
Secrets: use mounts, never ARG/ENV.

04 / Compose: use the project

BASH

```
docker compose config
docker compose up -d --build --wait
docker compose ps
docker compose logs -f
docker compose exec api sh
docker compose down
```

up: reconcile; **restart:** reuse config.
down: keeps named volumes; **-v:** deletes them (external volumes are excluded).
Startup order ≠ readiness. Use health + retry.
.env interpolation ≠ automatic injection.

05 / Network & data decisions

H:C = Host:Container.
`-p 8080:3000` → host uses port 8080.
Inside app: bind 0.0.0.0, not only localhost.
Peer: `service:container_port` on shared net.
EXPOSE is metadata, not publishing/security.

Durable data → volume.
Edit from host → bind mount.
Temporary memory-backed data → tmpfs.
Stop keeps the writable layer; remove loses it.
Mounts can hide files. Volumes are not backups.

06 / Diagnose & protect

```
docker inspect app
docker stats --no-stream app
docker system df -v
```

Read state/logs before deleting evidence.
137 suggests SIGKILL; check OOMKilled.
Use non-root USER; restrict daemon access.
Drop capabilities; avoid --privileged.
Limit memory/CPU/PIDs; bound logs.
Handle SIGTERM; probe health; test restore.
Unhealthy alone does not restart a container.

NEVER GUESS AT DESTRUCTIVE OPERATIONS

Check the Docker context, exact resource name and backup. A volume prune or compose down -v can delete data you still need.

OFFICIAL REFERENCES S03 S04 S12 S17 S21 S25 S28 S29 S39 S42

Full source index p18

Roadmap & final concept map

Practice until you can explain the boundary, reproduce the behavior and diagnose the failure.

LEARNING SEQUENCE

01 **Get a working Engine**
version, info, a disposable container

02 **Control a container**
run / exec / logs / stop / remove

03 **Build an image**
Dockerfile, context, cache, non-root

04 **Add network + data**
Service DNS, ports, volume restore

05 **Declare a stack**
Compose, health gates, secrets

06 **Ship and operate**
CI artifact, scan, limits, rollback

Practice checkpoints

Lab 1: run the p6 app, stop it gracefully, change its response and deploy the rebuilt image.

Lab 2: add a Redis client to the p9 API. Verify service DNS and bounded connection retries.

Lab 3: write data, replace the stack, back up the volume and prove a clean restore.

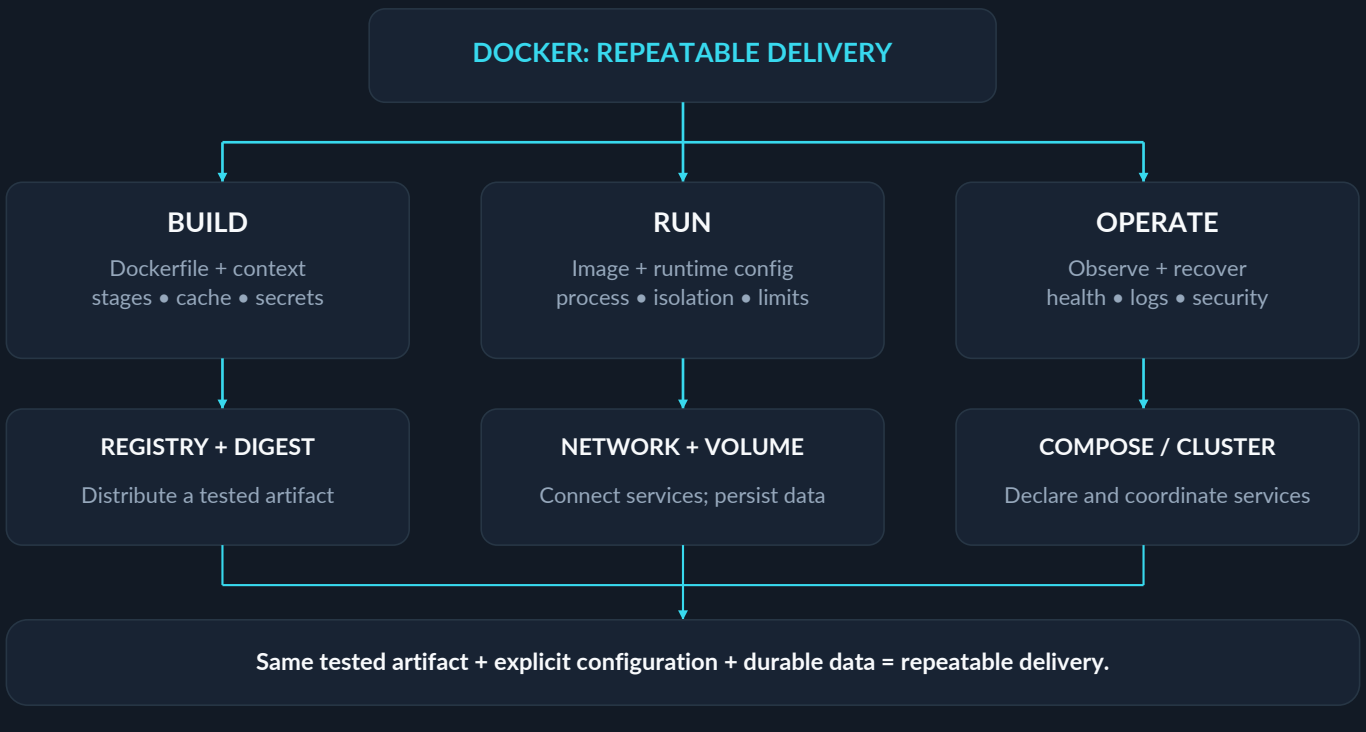
Lab 4: deliberately break a port, mount permission and health probe; diagnose without guesswork.

Next: add a CI pipeline with tests, multi-platform artifacts and a documented rollback. Then study orchestration.

READY FOR THE NEXT LEVEL?

Explain why a service can be running but unreachable, persistent but not backed up, and pinned but not yet patched.

THE FINAL CONCEPT MAP



OFFICIAL REFERENCES S01 S13 S17 S25 S35 S42 S49 S53

Full source index p18

Official source index

Primary documentation for checking details, flags, supported features and platform-specific behavior.

SCOPE & VERIFICATION

Reviewed **07 Sep 2026**. Examples target Linux containers, current Docker/BuildKit and the Compose Specification; shell commands use POSIX syntax. Image tags are teaching defaults, not immutable or vulnerability-free releases.
Commands and syntax were checked against the linked references. The Node example was exercised locally; Docker container builds were not executed in this environment.

SELECT A TITLE TO OPEN ITS OFFICIAL DOCUMENTATION